Informatika és távközlés ágazat

Synk

Készítette
Erdős Martin
Szedlák Krisztián
Makkai Krisztián
Szoftverfejlesztő és -tesztelő szak

Témavezető
Kerényi Róbert Nándor
oktató

Miskolc, 2026

Tartalomjegyzék

1. Bevezetés **4**

1.1. A projekt bemutatása és célja 4

1.2. Célközönség meghatározása 4

1.3. Alkalmazott technológiák és eszközök 5

2. Követelményanalízis **6**

2.1. Funkcionális követelmények 6

2.1.1. Webes felület (Next.js) 6

2.1.2. Mobil alkalmazás (React Native + Expo) 6

2.1.3. Adminisztrációs kliens (WPF) 6

2.2. Nem funkcionális követelmények 7

2.3. Használati esetek (Use Cases) 7

2.3.1. Kiemelt Use Case: Jegyvásárlás (UC-01) 7

3. Rendszertervezés **8**

3.1. Adatbázis terv 8

3.1.1. ER-diagram és kapcsolatok 8

3.1.2. Adatszótár és táblaleírások 9

3.2. Rendszerarchitektúra és komponensek 12

3.2.1. Általános architektúra 12

3.2.2. Komponensek és felelősségek 13

3.2.3. Skálázás és rendelkezésre állás 13

3.3. ORM (Object-Relational Mapping) 14

3.3.1. Tervezési döntések 14

3.3.2. Mapping és DTO stratégia 14

3.4. Entity Framework 14

3.4.1. DbContext és konfiguráció 14

3.4.2. Teljesítmény- és karbantartási praktikák 15

3.5. REST API végpontok tervezése 15

3.5.1. Erőforrás-orientált tervezés 15

3.5.2. Példa végpontok és viselkedés 15

3.5.3. Hiba- és státuszkódok 15

3.5.4. Biztonság és adatvédelem 16
3.6. UI/UX Tervek (Wireframes) 16

4. Megvalósítás **18**
4.1. Adatbázis réteg implementációja 18
4.2. REST Backend fejlesztése 18
4.2.1. Adatkezelés és Üzleti logika 19
4.2.2. Biztonság és Autentikáció 19
4.3. React Frontend fejlesztése 19
4.3.1. Komponens-architektúra 19
4.3.2. Reszponzív megoldások és Mobil nézet 19
4.4. WPF Desktop alkalmazás 19
4.4.1. MVVM architektúra és Adatkötés 20
4.4.2. Törzsadat-karbantartó modulok 20

5. Minőségbiztosítás és Tesztelés **21**
5.1. Unit tesztek (Egységtesztelés) 21
5.2. API tesztelés (Postman/Swagger) 21
5.3. Felhasználói felület és Reszponzivitás tesztelése 22
5.4. Hibajegyzék és javítások 22

6. Felhasználói útmutató **23**
6.1. Rendszerkövetelmények és Telepítés 23
6.2. A szoftver használata (Web/WPF) 23

7. Összegzés **25**
7.1. Projekt értékelése 25
7.2. Továbbfejlesztési lehetőségek 25

Mellékletek **28**

Irodalomjegyzék **29**

1. fejezet

Bevezetés

1.1. A projekt bemutatása és célja

A **Synk** egy innovatív, többplatformos koncertjegy-értékesítési ökoszisztéma. A projekt célja egy olyan integrált környezet létrehozása, amely a modern webes technológiák, az asztali alkalmazások ereje és a mobilis megoldások rugalmassága révén szolgálja ki a rendezvényszervezői piacot. A rendszer alapvető küldetése a jegyvásárlási folyamat transzparenssé tétele, a valós idejű készletkezelés és a biztonságos, digitális jegyfelhasználás biztosítása.

A Synk architektúrája négy fő pillérre épül: egy villámgyors vásárlói webfelületre, egy robusztus adminisztrációs asztali alkalmazásra, egy hordozható mobilalkalmazásra és az ezeket kiszolgáló központi API-ra.

1.2. Célközönség meghatározása

A szoftver célközönsége rétegzett, minden felhasználói típushoz dedikált felület tartozik:

- **Végfelhasználók (Vásárlók):** A Next.js felületen böngésznek, a React Native mobilalkalmazásban pedig tárolják és kezelik digitális jegyeiket.
- **Rendezvényszervezők:** A webfelületen / WPF kliensen keresztül végzik a komplex eseménymenedzsmentet és felhasználókezelést.
- **Helyszíni személyzet:** A mobilalkalmazás speciális funkcióival végzik a gyors, QR-kód alapú jegyérvényesítést.

Az automatikus tételbizonyítás egyik fontos megközelítése Herbrand nevéhez fűződik (1930). Herbrand algoritmust dolgozott ki olyan interpretáció előállítására, amely hamissá tehet egy formulát. Ugyanakkor, ha az illető formula logikailag igaz, tehát

nincs olyan interpretáció, amelyben hamissá válna, ez az algoritmus véges sok lépés után megáll. [**?**] Ez a logikai szigor alapvető a Synk jegyfoglalási algoritmusánál is, ahol el kell kerülni az olyan állapotokat, amelyek ellentmondáshoz (pl. túlfoglaláshoz) vezetnének.

1.3. Alkalmazott technológiák és eszközök

Lásd: 7.1 melléklet ábrát.

- **Adatbázis:** PostgreSQL [7] – A komplex relációs adatok és tranzakciók biztonságos kezeléséhez.
- **Backend:** RESTful API (ASP.NET Core) [3] – A központi üzleti logika kiszolgálója.
- **Webes kliens:** Next.js [5] – Szerveroldali rendereléssel támogatott React keretrendszer az optimális SEO és sebesség érdekében.
- **Mobil alkalmazás:** React Native (Expo) [6] – Cross-platform megoldás, amely lehetővé teszi a natív élményt iOS és Android eszközökön a digitális jegyek kezeléséhez.
- **Asztali kliens:** WPF (C#) [2] – Nagy teljesítményű adminisztrációs felület Windows környezetbe.
- **Gyorsítótár:** Redis [8] – Nagy teljesítményű in-memory kulcs-érték tároló a munkamenetek és adatok gyors eléréséhez.
- **Objektumtároló:** MinIO [9] – S3-kompatibilis nyílt forráskódú objektumtároló a médiafájlok és jegy PDF-ek kezeléséhez.

2. fejezet

Követelményanalízis

2.1. Funkcionális követelmények

A Synk rendszer funkcionális követelményei a tier-alapú jegyértékesítési modellre és a többrétegű kliensarchitektúrára fókuszálnak.

2.1.1. Webes felület (Next.js)

- **Események listázása:** A koncertek adatlapjának megjelenítése (leírás, dátum, fellépők).
- **Tier-alapú jegyválasztás:** A felhasználó választhat a különböző árkategóriájú és szolgáltatástartalmú jegytípusok (pl. Early Bird, Standard, VIP) közül a szabad készlet erejéig.
- **Jegylistázás:** Biztonságos jegykezelés, PDF formátumú jegyletöltés.

2.1.2. Mobil alkalmazás (React Native + Expo)

- **Jegyek kezelése:** A megvásárolt, különböző kategóriájú jegyek tárolása és vizuális megjelenítése.
- **QR-kódos validáció:** Egyedi, dinamikusan generált azonosítók megjelenítése a belépéshez.
- **Beléptető modul:** A személyzet számára elkülönített felület a jegyek szkennelésére és valós idejű érvénytelenítésére az API-n keresztül.

2.1.3. Adminisztrációs kliens (WPF)

A WPF alkalmazás kizárólag belső, adminisztratív célokat szolgál, közvetlen elérést biztosítva az üzleti adatokhoz:

- **Rendszerfelügyelet:** Felhasználó, és felhasználói jogosultságok kezelése és az adatbázis integritásának ellenőrzése.

2.2. Nem funkcionális követelmények

- **Tranzakcionális biztonság:** A PostgreSQL ACID tulajdonságait kihasználva garantálni kell, hogy a jegy-tierek darabszáma (stock) ne csökkenhessen negatív tartományba párhuzamos vásárlások esetén sem.
- **Keresőoptimalizálás (SEO):** A Next.js használatával biztosítani kell az eseményoldalak gyors indexelhetőségét.
- **Hordozhatóság:** A React Native kódalapnak minimális módosítással futnia kell mind iOS, mind Android eszközökön.
- **Adatvédelem:** A GDPR előírásoknak megfelelő titkosított adattárolás.

2.3. Használati esetek (Use Cases)

ID	Megnevezés	Aktor
UC-01	Jegyvásárlás adott kategóriában (Tier)	Vásárló
UC-02	Jegyérvényesítés mobil eszközzel	Személyzet
UC-03	Esemény és jegy-kvóták beállítása	Adminisztrátor
UC-04	Felhasználókezelés	Adminisztrátor

2.1. táblázat. A Synk platform használati esetei

2.3.1. Kiemelt Use Case: Jegyvásárlás (UC-01)

Leírás: A vásárló kiválasztja a kívánt jegy-tiert (pl. VIP), megadja a darabszámot, majd a rendszer ellenőrzi a szabad kontingenst az adatbázisban.

3. fejezet

Rendszertervezés

3.1. Adatbázis terv

3.1.1. ER-diagram és kapcsolatok

Az ER-diagram a rendszer fő entitásait és azok kapcsolatát mutatja be. A tervezés során a normalizáció 3NF-ig történik, ahol indokolt (teljesítmény- vagy lekérdezési okokból) denormalizáció alkalmazható cache-n vagy read-model rétegben.

Fő entitások és kapcsolatok (röviden):

- **User** — felhasználói fiókok: 1-N kapcsolat az **Order**-rel és 1-N a **Ticket**-tel (ha a jegyek fel vannak használóhoz rendelve). A User tartalmaz jogosultsági információkat (Role), amelyek az entitás részét képezik.
- **Event** — esemény: 1-N kapcsolat a **TicketType**-pal és az **Artist** / **Venue** entitásokkal.
- **TicketType** — egy eseményhez tartozó jegytípusok (árkategória, kvóta). 1-N kapcsolat a **Ticket**-tel; tartalmaz mezőket: price, maxSaleCount, saleStartTime.
- **Order** — vásárlási rendelés: 1-N kapcsolat az **OrderItem**-mel és a **Ticket**-tel. A rendelés státusza (Created, Processing, Completed, stb.) tükrözi a fizetési folyamat állapotát is.
- **OrderItem** — rendelési tételek: összekapcsolja a rendelést a jegytípussal (**TicketType**), tárolja a vásárolt mennyiséget és az egységárat a vásárlás pillanatában.
- **Ticket** — egyedi jegy entitás: tartalmazza a `ticketToken`-t, QR-kód és PDF elérhetőséget, státusza a kapcsolódó **TicketScan** adatokból és a `flaggedAsStolen` mezőből származtatható.
- **TicketScan** — jegyérvényesítések: tárolja, hogy melyik jegyet ki és mikor szkennelte be.

– **Artist**, **Venue** — eseményekhez kapcsolódó fellépők és helyszínek adatai.

Kapcsolati jellemzők:

– Kulcsok: minden táblának UUID alapú primer kulcs (Postgres: UUID) — uniformitás és globális ütközésmentesség.

– Idegen kulcsokra ON DELETE RESTRICT/NO ACTION alapértelmezés, ahol az integritás fontos; soft-delete megoldás (isDeleted + deletedAt) az üzleti adatok megőrzésére.

– Indexelés: gyakori lekérdezések alapján (Event.id, TicketType.eventId, Ticket.ticketToken, Order.userId), partiális indexek például csak aktív jegyekre.

3.1.2. Adatszótár és táblaleírások

A fontosabb táblák rövid leírása — oszlopnév (típus) — megjegyzések.

users

– id (UUID) — primer kulcs

– email (VARCHAR, unique) — bejelentkezéshez

– passwordHash (VARCHAR)

– firstName (VARCHAR)

– lastName (VARCHAR)

– role (VARCHAR) — Customer, Organizer, Administrator

– emailVerified (BOOLEAN)

– profilePictureUrl (VARCHAR)

– createdAt, updatedAt, deletedAt (TIMESTAMP WITH TIME ZONE) — soft delete

events

– id (UUID)

– name (VARCHAR)

– description (TEXT)

– startTime, endTime (TIMESTAMP WITH TIME ZONE)

- venueId (UUID) — FK -> venues.id
- artistId (UUID) — FK -> artists.id
- thumbnailUrl (VARCHAR)
- totalCapacity (INT)
- ticketMaxScanCount (INT)
- gateStaffCode (VARCHAR) — beléptető személyzet kódja
- createdAt, updatedAt

ticket_types

- id (UUID)
- eventId (UUID) — FK -> events.id
- name (VARCHAR)
- price (NUMERIC(10,2))
- maxSaleCount (INT) — teljes kvóta
- saleStartTime, saleEndTime (TIMESTAMP WITH TIME ZONE)
- createdAt, updatedAt

orders

- id (UUID)
- userId (UUID) — FK -> users.id
- billingAddressId (UUID) — FK -> billing_addresses.id
- status (VARCHAR) — Created, Processing, Completed, Failed, Cancelled, Refunded
- createdAt, updatedAt

tickets

- id (UUID)
- userId (UUID) — FK -> users.id
- eventId (UUID) — FK -> events.id

- ticketTypeId (UUID) — FK -> ticket_types.id
- orderId (UUID) — FK -> orders.id
- ticketToken (VARCHAR, unique)
- qrCodeUrl (VARCHAR)
- pdfUrl (VARCHAR)
- flaggedAsStolen (BOOLEAN)
- createdAt, updatedAt

venues

- id (UUID)
- name (VARCHAR)
- description (TEXT)
- country, city, address (VARCHAR)
- capacity (INT)
- isAdultOnly (BOOLEAN)
- createdAt, updatedAt

artists

- id (UUID)
- name (VARCHAR)
- description (TEXT)
- spotifyUrl (VARCHAR)
- profilePictureUrl (VARCHAR)
- createdAt, updatedAt

billing_addresses

- id (UUID)
- userId (UUID) — FK -> users.id
- firstName, lastName (VARCHAR)

- email (VARCHAR)
- phoneNumber (VARCHAR)
- country, city, address, zipCode (VARCHAR)
- createdAt, updatedAt

order_items

- id (UUID)
- orderId (UUID) — FK -> orders.id
- ticketTypeId (UUID) — FK -> ticket_types.id
- quantity (INT)
- price (NUMERIC(10,2)) — egységár a vásárláskor
- createdAt, updatedAt

ticket_scans

- id (UUID)
- ticketId (UUID) — FK -> tickets.id
- scannedByUserId (UUID) — FK -> users.id
- scannedAt (TIMESTAMP)
- createdAt, updatedAt

3.2. Rendszerarchitektúra és komponensek

3.2.1. Általános architektúra

A Synk többrétegű, mikroserviz-szerű logikát követ, de a vizsgaprojekt érdekében monolitikus, modulárisan felépített backend (ASP.NET Core) [3] került tervezésre, amelyet a következő logikai rétegekre bontunk:

- **API réteg** — REST API, kontroller réteg, DTO-kon keresztül kommunikál.
- **Service/Business réteg** — üzleti szabályokat, tranzakciókezelést, validációt tartalmazza.

- **Repository/Domain réteg** — adatbázis műveletek, Unit of Work, EF DbContext kezelése.
- **Client réteg** — Next.js webapp [5], React Native mobilapp [6], WPF admin kliens [2].
- **Perzisztencia** — PostgreSQL adatbázis [7], opcionálisan CQRS read-model cache (Redis) olvasási optimalizációhoz.
- **Aszinkron feldolgozás** — background jobok (pl. jegy-generálás, szinkronizáció külső szolgáltatóval), RabbitMQ vagy beépített háttérfeladatok.
- **FileStorage** — profilképek, jegy PDF-ek tárolása S3-kompatibilis tárolón (MinIO).

3.2.2. Komponensek és felelősségek

- **API Gateway / Reverse Proxy** — kérelem irányítás, TLS termináció (Traefik).
- **Auth Service** — Session token alapú authentikáció, refresh token kezelés; jogosultságok RBAC modellben.
- **Mobil App** — alacsony késleltetésű jegyvalidáció; jegykezelés

Deployment szempontok: konténerizáció (Docker), környezetspecifikus konfiguráció (konfigurációk titkos kezelése: Vault/Environment variables), CI/CD pipeline (build, migrációk, smoke tesztek).

3.2.3. Skálázás és rendelkezésre állás

- Vertikális skálázás a kezdeti fázisban; horizontális skálázás stateless service-eknél (API), állapotot DB/Redis tárolja.
- Adatbázis: replikáció (read-replica) olvasási terhelés leválasztására; master node tranzakciókhoz.
- Failover tervek és backup stratégia: napi teljes mentés, bináris WAL archiválás a gyors helyreállításhoz.

3.3. ORM (Object-Relational Mapping)

3.3.1. Tervezési döntések

Az ORM használata (Entity Framework Core) [4] gyors fejlesztést és típusbiztonságot nyújt. Az EF-et a következő szabályok mentén használjuk:

- **Explicit mapping** — minden entitást Fluent API-val konfigurálunk, elkerülve implicit konvenciókra való túlzott támaszkodást.
- **Azonnali vs késleltetett betöltés** — alapértelmezettként explicit Include használata nagy lekérdezéseknél; Lazy Loading csak jól ellenőrzött helyeken, hogy elkerüljük a N+1 problémákat.
- **Tranzakciókezelés** — minden kritikus művelet (pl. jegyvásárlás) explicit tranzakcióba kerül: DbContext.Database.BeginTransactionAsync() + commit/rollback.
- **Concurrency** — Optimistic Concurrency: minden fontos entitás (pl. TicketType) tartalmaz ‘row_version‘(bytea / xmin / timestamp) mezőt, EF ConcurrencyTokenként kezelve; konfliktus esetén visszadobott DbUpdateConcurrencyException kezelése és retry logika alkalmazása.

3.3.2. Mapping és DTO stratégia

A belső domain entitások nem kerülnek közvetlenül a kliens felé. DTO-kat és automapper profilt használunk a lehetséges túlzott adatkinyerés elkerülésére, valamint a visszaadott JSON szerkezet stabilitásának biztosítására.

3.4. Entity Framework

3.4.1. DbContext és konfiguráció

A ‘ApplicationDbContext‘a per-request scope-ban kerül regisztrálásra (AddDbContext). A konfiguráció alapelvei [4]:

- **Fluent API** minden táblához: kulcsok, indexek, check constraint-ek definiálása kód alapú migrációk során.
- **Migrations** — kódból létrehozott, verzionált migrációk használata (dotnet ef migrations add‘), migráció futtatása deployment pipeline részeként.
- **Query optimization** — AsNoTracking() read-only lekérdezésekhez, projection (Select új DTO-kra) a felesleges adatátvitel csökkentésére.

- **Raw SQL** — csak akkor alkalmazzuk, ha a LINQ nem tud hatékony, optimalizált SQL-t generálni (pl. komplex aggregációk), és az SQL-injection elleni védelem biztosított paraméterezéssel.

3.4.2. Teljesítmény- és karbantartási praktikák

- N+1 probléma elkerülése: Eager loading (Include) és batch lekérések, illetve explicit Select-projekciók.
- Indexek felülvizsgálata ‘EXPLAIN ANALYZE‘alapján, és migrációval indexek hozzáadása/eltávolítása.
- Long-running query-k monitorozása és timeoutok beállítása (CommandTimeout).

3.5. REST API végpontok tervezése

3.5.1. Erőforrás-orientált tervezés

Az API RESTful elveket követ: erőforrások URL-jei (pl. ‘/api/events/id‘, ‘/api/events/id/ticket-types‘) következetesek és intuitívak. Verziózás: Accept header-ben a jövőbeli visszafelé kompatibilitás miatt.

3.5.2. Példa végpontok és viselkedés

- GET /api/events‘— lista + pagination + filterek (dateRange, artistId) + sort.
- GET /api/events/id‘— részletes esemény adatok (ticketTypes nélkül vagy opcionálisan ?include=ticketTypes).
- ‘POST /api/orders/purchase‘— jegyvásárlás. Fontos: idempotencia kulcs támogatása (Idempotency-Key header) a duplikált fizetések elkerülésére. A végpont atomikus tranzakciót indít, ellenőrzi a kvótát és lefoglalja a jegyeket, majd kezdeményezi a fizetést.
- ‘PATCH /api/tickets/id/validate‘— jegy érvényesítés beléptető appból, audit log készítése minden érvényesítésről.

3.5.3. Hiba- és státuszkódok

Következetes HTTP státuszkódok és hibaválaszok:

- 200 OK — sikeres lekérés
- 201 Created — sikeres létrehozás

- 202 Accepted — aszinkron folyamat elfogadva (pl. külső fizetés feldolgozása)
- 400 Bad Request — validációs hibák; válasz részletes hibaobjektummal (‘field‘, ‘message‘)
- 401 Unauthorized, 403 Forbidden — auth/authz hibák
- 404 Not Found — nem létező erőforrás
- 409 Conflict — versenyhelyzet vagy idempotencia ütközés (pl. quota insuf.)
- 500+ — szerver oldali hibák (monitoring és alerting generálása)

3.5.4. Biztonság és adatvédelem

Minden érzékeny művelethez HTTPS kötelező. Session tokenek short-lived access token + long-lived refresh token mechanizmussal.

A jegyvásárlás kritikus pont; ajánlott megközelítések:

- **Pesszimista blokkolás**: SELECT ... FOR UPDATE a kvóta soron, ha a konkurencia nagyon magas és a tranzakciók rövidek.
- **Optimistic concurrency**: row_version ellenőrzés; ha verseny van, a tranzakció visszadobásra kerül és a kliens retry mechanizmussal értesül.
- **Idempotencia**: belső vásárlás egységesen kezelhető idempotencia kulccsal, hogy duplikáció elkerülhető legyen.

3.6. UI/UX Tervek (Wireframes)

A wireframe-ek és UI specifikáció célja a felhasználói folyamatok minimalizálása és a hibalehetőségek csökkentése.

Fő elvek:

- **Egyszerűsített vásárlási folyamat** — 3 lépés: (1) esemény kiválasztás, (2) jegytípus & darabszám, (3) fizetés és visszaigazolás/QR megjelenítés.
- **Responszív kialakítás** — Next.js + CSS-in-JS (pl. Tailwind/Emotion) a komponensek mobil/desktop változatára; WPF admin felület esetén rugalmas grid és skálázható vezérlők.
- **Átlátható hibakezelés** — valós idejű validáció (pl. darabszám > quota figyelmeztetés), inline hibauzenetek és globális toast értesítések.

- **Accessibility (a11y)** — releváns aria attribútumok, kontraszt, tab-navigáció, képernyőolvasó támogatás.
- **Beléptetés UX** — Mobil alkalmazás: minimalisztikus képernyő nagy, egyértelmű validáció gombokkal.

Wireframe forgatókönyvek:

1. **Vásárlói út**: keresés -> esemény lap -> jegyválasztó modal (valós időben elérhető mennyiség) -> fizetési űrlap -> visszaigazolás + jegy PDF/QR.
2. **Admin esemény létrehozás**: űrlap több lépcsőben (alapadatok, jegytípusok, kvóták, publikálás); előnézeti nézet a publikus oldallal.
3. **Beléptetési út**: kereső / QR scanner -> jegy megtalálása -> státusz ellenőrzés -> érvényesítés + audit log.

4. fejezet

Megvalósítás

4.1. Adatbázis réteg implementációja

Az adatbázis réteg megvalósítása során az Entity Framework Core (EF Core) ORM-re támaszkodtunk, amely Code-First megközelítéssel biztosítja a séma és a C# entitások közötti szinkront. Az adatbázis-műveletek absztrakciójára a **Repository Pattern**-t alkalmaztuk, amely elválasztja az üzleti logikát a közvetlen adatkezelési implementációtól.

A migrációk kezelése a .NET CLI eszközeivel történik, biztosítva a séma verziókövetését és a környezetek közötti konzisztenciát. A teljesítmény optimalizálása érdekében a lekérdezéseknél `AsNoTracking()` hívásokat használunk a csak olvasható adatokhoz, és explicit indexeket definiáltunk a gyakran keresett mezőkhöz (pl. `TicketToken`, `EventId`).

4.2. REST Backend fejlesztése

A backend fejlesztésekor az **Onion (Clean) Architecture** elveit [1] követtük. Ez a struktúra négy fő rétegre oszlik:

- **Domain:** Tartalmazza az alapvető üzleti entitásokat és interfészeket.
- **Application:** Itt találhatók a szolgáltatások (`OrderService`, `EventService`), amelyek a konkrét üzleti folyamatokat (pl. jegyvásárlási logika released capacity pooling-gal) implementálják.
- **Infrastructure:** A technikai részletek, mint az EF Core DbContext [4], a Redis cache és a MinIO fájlkezelés megvalósítása.
- **API:** Az ASP.NET Core kontroller réteg [3], amely a HTTP kéréseket fogadja és Scalar alapú dokumentációt biztosít.

4.2.1. Adatkezelés és Üzleti logika

Az adatátvitelhez DTO (Data Transfer Object) mintát használunk, elkerülve a belső entitások közvetlen expozícióját. A bejövő kérések validálását a **FluentValidation** könyvtár végzi, amely deklaratív módon kényszeríti ki az üzleti szabályokat (pl. jegyvásárlási limit, dátumok validitása). A tranzakciókezelés atomi módon biztosítja, hogy egy hiba esetén ne maradjanak inkonzisztens adatok (pl. levont kvóta jegy-létrehozás nélkül).

4.2.2. Biztonság és Autentikáció

A rendszer biztonságát munkamenet-alapú (session) tokenek és JWT alapú megoldások kombinációja garantálja. A felhasználók autentikációja után a backend egy biztonságos tokent generál, amelyet a kliensek (WPF esetén SecureStore, Web esetén HttpOnly cookie-k) tárolnak. A jogosultságkezelés **RBAC (Role-Based Access Control)** modellen alapul, megkülönböztetve az általános vásárlókat, a rendezvényszervezőket és a beléptető személyzetet.

4.3. React Frontend fejlesztése

A vásárlói felület Next.js keretrendszerben [5] készült, kihasználva a React 19 és a Next.js App Router adta legmodernebb lehetőségeket.

4.3.1. Komponens-architektúra

A frontend moduláris felépítésű, ahol a komponensek logikailag elkülönülnek (pl. jegyválasztó, eseménykártya, profilkezelő). Az állapotkezelést a React beépített `Context API`-ja és a `server-side fetching` megoldások biztosítják. Az API kommunikációt egy centralizált Axios kliens végzi, amely automatikusan kezeli a tokenek transzferét.

4.3.2. Reszponzív megoldások és Mobil nézet

A stílusozáshoz a legújabb **Tailwind CSS 4.0**-át [10] használtuk, amely utility-first megközelítésével gyors és konzisztens fejlesztést tesz lehetővé. A felület teljesen reszponzív, optimalizált megjelenést nyújtva asztali számítógépeken, tableteken és mobilböngészőkben egyaránt.

4.4. WPF Desktop alkalmazás

Az adminisztrációs feladatok elvégzésére szolgáló WPF kliens a Microsoft modern .NET környezetére [3] épül.

4.4.1. MVVM architektúra és Adatkötés

Az alkalmazás az **MVVM (Model-View-ViewModel)** mintát [2] követi, amely éles választóvonalat húz a felhasználói felület és a mögöttes üzleti logika közé. Az adatkötés (Data Binding) segítségével a UI dinamikusan frissül a ViewModel állapotváltozásaira, csökkentve a manuális vezérlőkezelést.

4.4.2. Törzsadat-karbantartó modulok

A WPF kliens dedikált modulokat tartalmaz az események, helyszínek és fellépők kezelésére. A backenddel való kommunikációt egy thread-safe `ApiClient` wrapper végzi, amely kezeli a hibatűrő hálózati kéréseket és a JSON deszerializációt.

5. fejezet

Minőségbiztosítás és Tesztelés

A rendszer megbízhatóságának és stabilitásának alapja a többszintű tesztelési stratégia, amely a fejlesztési folyamat minden fázisát lefedi.

5.1. Unit tesztek (Egységtesztelés)

A backend szolgáltatások (`Synk.Application`), az adattárolási logika (`Synk.Infrastructure`), valamint a WPF alkalmazás ViewModel rétege egységtesztelésére az **xUnit** keretrendszert alkalmaztuk. A külső függőségeket (például adatbázis-interfészek, cache szolgáltatás) mockolással szigeteltük el, biztosítva a tesztek gyorsaságát és izoláltságát. A frontend egységtesztekhez a Jest tesztkeretrendszert használtuk, amelyet komponensek és oldalak tesztelésére alkalmaztunk.

- **Service tesztek:** Validálják az üzleti logikát (pl. `OrderServiceTests` ellenőrzi a jegykvóták és a fizetési folyamat helyességét).
- **Validator tesztek:** A `FluentValidation` szabályokat tesztelik a bejövő kérések integritásának garantálása érdekében.
- **Mapper tesztek:** Biztosítják, hogy az entitások és DTO-k közötti leképezés adatvesztés nélkül történjen meg.

5.2. API tesztelés (Postman/Swagger)

Az integrációs tesztelés során a végpontok működését valós környezetben vizsgáltuk. A backend beépített **Scalar** (OpenAPI) felülete lehetővé teszi az interaktív tesztelést közvetlenül a böngészőből. Emellett automatizált integrációs tesztek (`EventControllerIntegrationTes` `OrdersControllerIntegrationTests`) futnak, amelyek tesztadatbázist használva ellenőrzik a teljes kérés-válasz ciklust.

5.3. Felhasználói felület és Reszponzivitás tesztelése

A frontend tesztelése során a manuális és eszközalapú ellenőrzésekre fókuszáltunk:

- **Reszponzivitás:** A böngészők fejlesztői eszközeivel szimuláltuk a különböző kijelzőméreteket (mobil, tablet, desktop).
- **Cross-browser tesztelés:** Chrome, Firefox és Safari böngészőkben ellenőriztük a megjelenés konzisztenciáját.
- **Mobil app tesztelés:** Az Expo segítségével fizikai eszközökön teszteltük a QR-szkennelést és a navigációs folyamatokat.

5.4. Hibajegyzék és javítások

A fejlesztés során felmerült hibákat rendszerezetten kezeltük. A kritikus problémákat (pl. concurrency ütközések a jegyvásárlásnál) prioritással javítottuk, míg a kisebb UI anomáliákat iteratív módon korrigáltuk a fejlesztési ciklusok végén. A hibák elhárítását követően minden esetben regressziós tesztelést végeztünk a kapcsolódó modulokon.

6. fejezet

Felhasználói útmutató

Ez a fejezet áttekintést nyújt a Synk ökoszisztéma telepítéséről és napi használatáról.

6.1. Rendszerkövetelmények és Telepítés

A rendszer futtatásához az alábbi környezeti feltételek szükségesek:

- **Backend:** .NET 10 SDK, PostgreSQL 16+, Redis.
- **Web/Mobil:** Node.js v20+, yarn, Expo Go app.
- **Desktop:** Windows 10/11, .NET Runtime.

Telepítési lépések:

1. A projektek klónozása a GitHub repositoryk-ból: `https://github.com/synkhu/`.
2. Az adatbázis és a fájltároló (MinIO) elindítása Docker Compose segítségével.
3. A backend függőségek telepítése és a migrációk futtatása: `./Scripts/apply-migrations.sh`.
4. A webes frontend indítása: `yarn install && yarn dev` a `Synk-Frontend` mappában.
5. A mobilalkalmazás futtatása az applikáció telepítőn keresztül.

6.2. A szoftver használata (Web/WPF)

Vásárlói folyamat (Web): A felhasználók a főoldalon böngészhetik az eseményeket, a **"Jegyvásárlás"** gombra kattintva választhatják ki a kívánt kategóriát. A sikeres vásárlás után a jegyek a profilukban és a mobilalkalmazásban azonnal megjelennek.

Adminisztrációs folyamat (WPF): Az operátorok a **Dashboard** segítségével valós időben követhetik az eladásokat, új eseményeket hozhatnak létre, és kezelhetik a felhasználói jogosultságokat. A bejelentkezéshez külön operátori fiók szükséges.

Beléptetés (Mobil): A helyszíni személyzet a mobilalkalmazás **"Scan"** funkcióját használva validálja a jegyeket. A sikeres beolvasáskor a rendszer azonnal `Used` státuszba állítja a jegyet, megakadályozva a többszöri belépést.

7. fejezet

Összegzés

A Synk projekt sikeresen demonstrálta egy modern, elosztott architektúrára épülő koncertjegy-értékesítési rendszer megvalósíthatóságát.

7.1. Projekt értékelése

A fejlesztés során sikerült integrálni a .NET 10 [3], a Next.js [5] és a React Native [6] technológiákat egy egységes ökoszisztémába. A rendszer alapvető funkciói – a jegyvásárlástól a beléptetésig – stabilan és biztonságosan működnek. A PostgreSQL [7] alapú tranzakciókezelés és a Redis cache használata nagy terhelés mellett is biztosítja a konzisztenciát és a gyors választidőt. A projekt bebizonyította, hogy a modern webes és mobil technológiák kombinálásával professzionális felhasználói élmény nyújtható a rendezvényszervezők és a vásárlók számára egyaránt.

7.2. Továbbfejlesztési lehetőségek

Bár a projekt alapvető célkitűzéseit elértük, több irányban is látunk lehetőséget a bővítésre:

- **Fizetési gateway-ek integrálása:** Valós Stripe vagy PayPal fizetési rendszerek bekötése a szimulált folyamatok helyett.
- **Valós idejű értesítések:** WebSockets (SignalR) használata a jegyek fogyásának és az eseményváltozásoknak a követésére.
- **Analitika:** Fejlettebb statisztikai modulok készítése a rendezvényszervezők részére a WPF kliensen belül.
- **Offline mód:** A mobilalkalmazás fejlesztése, hogy korlátozott hálózati elérés esetén is képes legyen a jegyek validálására.

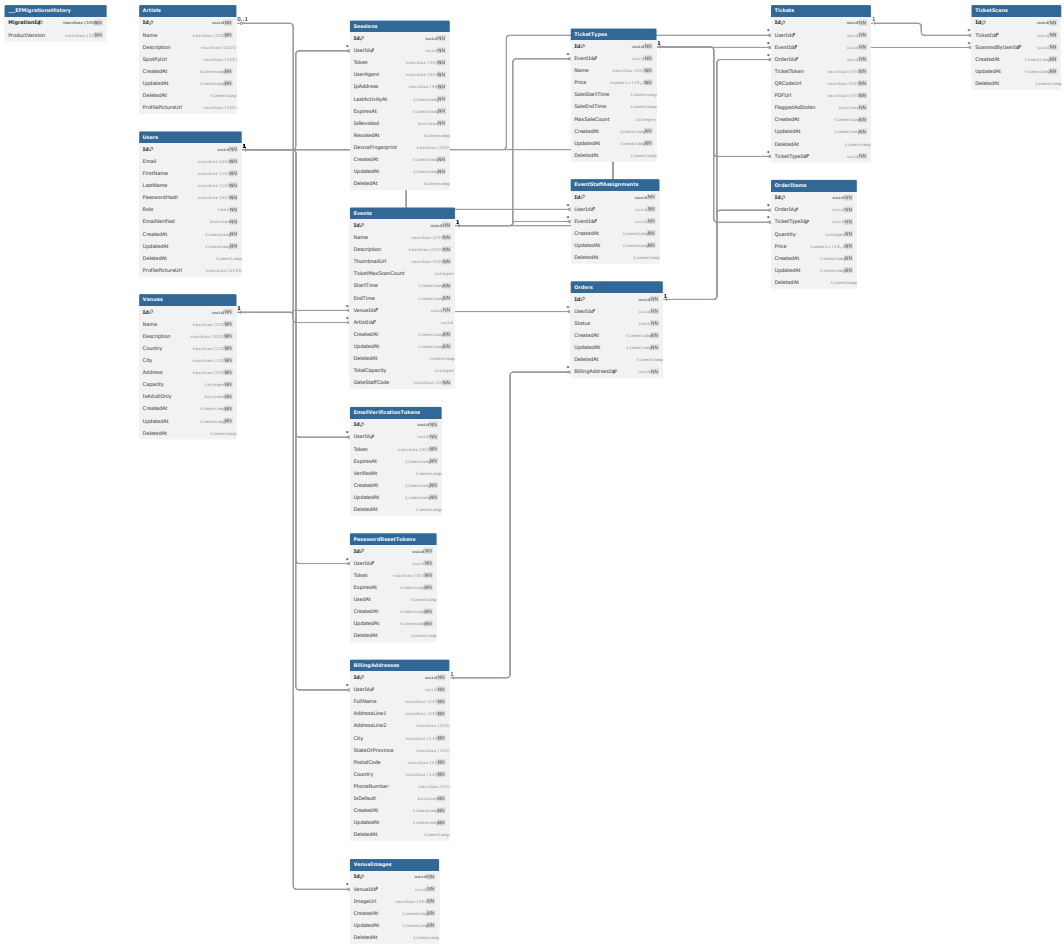
A jövőben a Synk továbbfejleszthető egy teljes körű rendezvénymenedzsment platformmá, amely a jegyértékesítésen túl a marketinget és a CRM funkciókat is integrálja.

Köszönetnyilvánítás

Szeretnénk köszönetet mondani mindazoknak, akik segítették a vizsgaremek létrejöttét. Külön köszönettel tartozunk Kerényi Róbert Nándor (Backend programozás), Négyesi Gábor (Frontend programozás), Deák Csaba (Asztali alkalmazás), Kasza László Róbert (Adatbázis kezelés) és Lázár Zsolt (Adatbázis kezelés) tanár úr részére, akik szakmai tudásukkal, útmutatásukkal és folyamatos támogatásukkal nagyban hozzájárultak munkánk sikeres megvalósításához. Hálásak vagyunk mindannyiuk támogatásáért a teljes folyamat során.

„De ha benned van a forrás, akkor legyen felette kút”
— Beton.Hofi: *BETON.HOF1* (INTERREGNUM)

Mellékletek



7.1. ábra. Adatbázis ábra

Irodalomjegyzék

[1] Jeffrey Palermo: *The Onion Architecture: part 1*, 2008. https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

[2] John Gossman: *Introduction to Model/View/ViewModel pattern for building WPF apps*, 2005.

[3] Microsoft: *.NET 10 and ASP.NET Core Documentation*, 2025. https://learn.microsoft.com/en-us/dotnet/

[4] Microsoft: *Entity Framework Core Documentation*, 2025. https://learn.microsoft.com/en-us/ef/core/

[5] Vercel: *Next.js Documentation*, 2025. https://nextjs.org/docs

[6] Meta: *React Native · A framework for building native apps using React*, 2025. https://reactnative.dev/docs/getting-started

[7] PostgreSQL Global Development Group: *PostgreSQL 16 Documentation*, 2023. https://www.postgresql.org/docs/16/index.html

[8] Redis: *Redis Documentation*, 2025. https://redis.io/docs/

[9] MinIO: *MinIO High Performance Object Storage*, 2025. https://min.io/docs

[10] Tailwind Labs: *Tailwind CSS Documentation*, 2025. https://tailwindcss.com/docs